GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unitat 02 — Introducció a Java
Bloc 1

1r CFGS DAW · Programació

Autores: Sergi García, Guillermo Garrido y Alfredo Oltra

En aquesta sessió

Del zero a "Hola, món!" passant per la màquina de café

1. Què és Java i per què funciona igual a tot arreu (JVM, bytecode).
2. La trilogia del café: **JVM, JRE i JDK** — qui és qui.
3. Instal·lació del **OpenJDK** i verificació amb `java -version`.
4. El teu primer programa: `HolaMundo` diseccionat línia a línia.
5. El **depurador**: visió de raigs X per al teu codi.

> ℹ️ Cap experiència prèvia necessària: només un editor de text i curiositat.

L'ordinador és molt llest, però té zero iniciatives

No fa **res** fins que no li dones ordres precises, en orde i sense ometre cap detall. Programar és precisament això: donar-li eixes ordres en un idioma que entenga.

Per a això necessitem tres coses: un **entorn de desenrotllament**, un **llenguatge** i moltes ganes de compilar.

> ℹ️ Aquesta unitat et deixa a la porta del castell: JDK instal·lat, primer programa escrit i depurador a punt.

Què és Java?

ℹ️ Java és un llenguatge que s'executa dins d'una **màquina virtual (JVM)**: corre igual en Windows, Linux o macOS.

Escrius una vegada i el mateix programa funciona en qualsevol lloc.

La JVM és l'intèrpret que tradueix les teues ordres a l'idioma concret de cada màquina.

Write once, run anywhere

```
public class Missatge {
    public static void main(String[] args) {
        System.out.println("☕ Escriu-ho una vegada, corre en qualsevol lloc");
    }
}
```

D'on ix Java?

1. **1995**: naix en **Sun Microsystems**, inspirat en la màquina de café de l'oficina (d'ací el logo i el nom).
2. La idea: un llenguatge que funcione en **qualsevol dispositiu**, sense importar el sistema operatiu ni el maquinari.
3. La solució: no programar per a l'ordinador, sinó per a una **màquina virtual** que l'ordinador simula.

> ℹ️ 💡 **Dada freak:** el llenguatge es deia primer *Oak* (roure), per un arbre que es veia des de l'oficina. Van canviar-lo per motius de marca registrada.

El viatge del teu codi: el bytecode

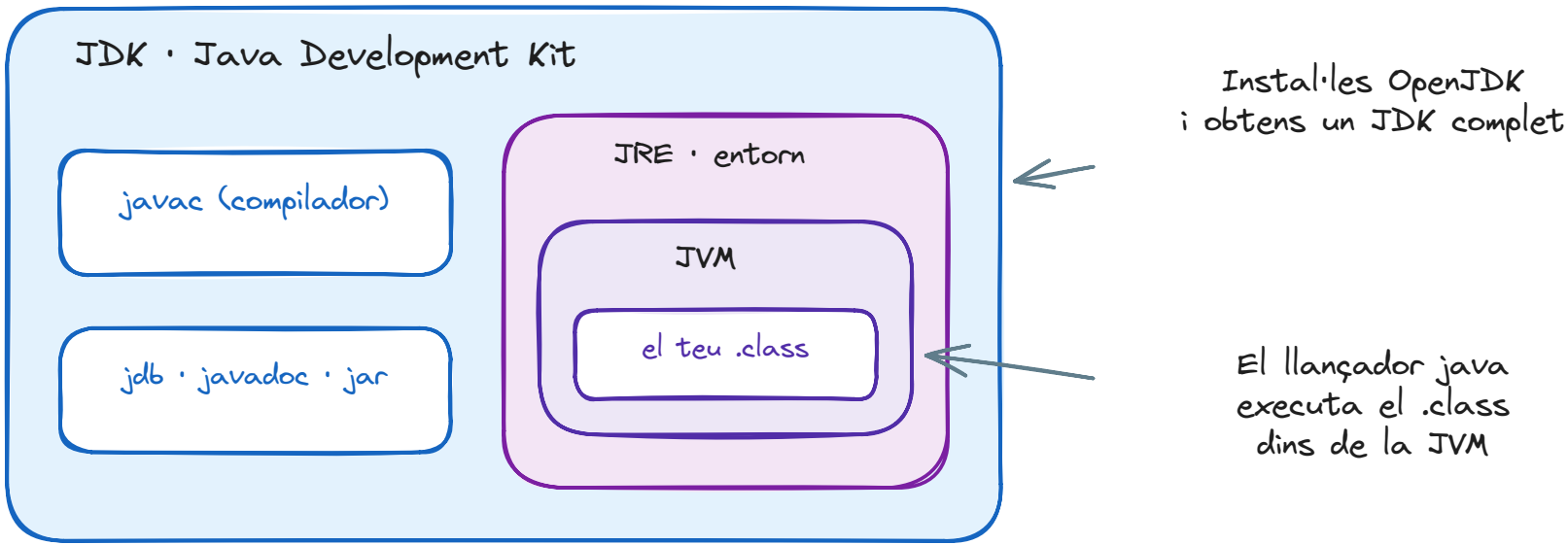
Quan escrius Java no escrius instruccions per a la CPU: les escrius per a la **JVM**.

1. Escrius codi font en un arxiu `.java`.
2. El compilador **javac** el tradueix a **bytecode**: un idioma intermedi que entén la JVM (`.class`).
3. La **JVM** llig el bytecode i l'executa en la teua màquina concreta.

```
$ el teu codi (.java) --javac⟶ bytecode (.class) --JVM⟶ ¡s'executa!
```

ℹ️ El `.class` és el mateix per a totes les plataformes: només canvia la JVM, no el teu programa.

Qui conté què: JDK ⊃ JRE ⊃ JVM



ℹ️ Un JDK conté tot: si instal·les **OpenJDK** ja tens `javac`, el llançador `java`, el depurador `jdb` i la JVM dins.

☕ La trilogia del Café: qui és qui

Concepte	Definició	Analogia
JVM	La màquina que executa el *bytecode*	La màquina de café: funciona igual en qualsevol lloc
JRE	Tot el necessari per a *executar* Java	La cafeteria sencera: màquina, gots, sucre...
JDK	Tot el necessari per a *crear* programes	El kit per a muntar la teua cafeteria: grans, molinet i manual de barista

⚠️ El **JDK inclou el JRE**: instal·lant el JDK tens les dues coses. Instal·lar només el JRE et permet executar programes, però no crear-los.

ℹ️ **JDK = el ganivet del xef · JRE = el plat servit**

El ritu de cada programa Java

Cafe.java — JDK 21

```java
// Comentari: el codi s'explica sol
public class Cafe {
  public static void main(String[] args) {
    System.out.println("☕ ¡Café preparat!");
  }
}
```

1. El **JDK** compila el codi a bytecode.
2. El **JRE** ho passa per la JVM.
3. Tachán! Café (text) en la teua pantalla.

ℹ️ Aquesta estructura (classe + `main`) és l'esquelet de **tot** programa Java del curs.

El ritu, dibuixat



✅ ✍️ **javac** (JDK) compila el codi font a **bytecode** · ⚙️ **java** (JVM) l'executa: ☕ text a la consola. Retén el tub: `editar → compilar → executar`.

Per què Java segueix viu tants anys després?



Multiplataforma: mòbils, servidors, caixers... fins i tot la rentadora intel·ligent.



Món empresarial: banca, assegurances i logística fa dècades que l'usen.



Android: el llenguatge històric del sistema operatiu més estès del món.



Comunitat enorme: el teu error probablement ja el va resoldre algú fa deu anys.

✅ És exigent, i això et fa millor: t'obliga a ser ordenat i a escriure codi més net.

Comprovació ràpida ☕

? Quina és la diferència entre JDK i JRE en una frase?

? Què fa el compilador `javac` amb el teu codi `.java`?

? Per què el logo de Java és una tassa fumejant?

1. El JDK crea programes (inclou el compilador); el JRE només els executa — el JDK conté el JRE. · **2.** El tradueix a bytecode (un arxiu `.class`) que la JVM pot executar. · **3.** Per la màquina de café de l'oficina de Sun: «escriu una vegada, corre en qualsevol lloc».

Quin JDK instal·le?

1. **OpenJDK** — el projecte de referència, lliure i de codi obert. **La nostra opció recomanada** (més senzill en Linux).
2. **Eclipse Temurin** — distribució excel·lent basada en OpenJDK, lliure i mantinguda per la fundació Eclipse.
3. **Oracle JDK** — la versió comercial, vàlida en entorns empresarials (més senzill d'instal·lar en Windows).

ℹ️ **La versió:** tria l'última **LTS** (suport a llarg termini): 17, 21 o superior. Tot el curs funciona igual en totes elles.

ℹ️ **El criteri:** lliure i de referència → `openjdk.org`. No t'obsessions amb el número exacte.

Instal·la i verifica

1. **Instal·la** OpenJDK amb els valors per defecte. En Windows, marca l'opció d'afegir el JDK al **PATH** si l'instal·lador t'ho ofereix.
2. **Verifica** obrint una terminal i comprovant l'executor i el compilador:

```
> java -version
openjdk version "21.0.2" 2024-01-16
OpenJDK Runtime Environment (build 21.0.2+13-LTS)
> javac -version
javac 21.0.2
```

✅ Si veus una eixida pareguda: enhorabona, tens poders de compilació actius.

Què és el PATH?

El **PATH** és la llista de carpetes on el sistema operatiu busca els comandaments que escrius en la terminal.

✅ Amb `...\jdk-26\bin` dins del PATH: escrius `java` i el sistema el troba a la primera.

⚠️ Sense eixa configuració: *«java no es reconeix com un comandament intern o extern»* — el sistema no sap on està el teu JDK.

Remei ràpid en Windows:

1. Busca «Editar les variables d'entorn del sistema».
2. Afig la carpeta `bin` del teu OpenJDK a la variable `Path`.

Dos comandaments, dos papers

javac — el compilador

Converteix el teu codi font `.java` en bytecode `.class`.

```
> javac HolaMundo.java
(apareix HolaMundo.class)
```

java — l'executor

Arranca la **JVM** per a executar el bytecode que has compilat.

```
> java HolaMundo
¡Hola, Mundo!
```

ℹ️ **Es necessiten els dos**: primer `javac` tradueix, després `java` posa en marxa. Els veuràs treballar junts tot el curs.

L'IDE: la teua navalla suïssa

IDE / Editor	Punts forts
VS Code	L'opció recomanada: lleuger, modern i amb l'extensió *Extension Pack for Java* és un entorn complet
IntelliJ IDEA (Community)	El favorit del sector professional; autocompletat bestial, però una mica més pesat
NetBeans	Simple i oficial d'Oracle, perfecte per a començar
Eclipse	Clàssic, molt usat en empreses, un pèl més dens

ℹ️ Un IDE reuneix: **editor** amb autocompletat + **compilador i executor** amb un botó + **depurador** + **gestió de projectes**. Tots valen: l'IDE és una ferramenta, no l'objectiu.

Exemple guiat: primer projecte en VS Code

1. `Ctrl + Shift + P` → *Java: Create Java Project* → *No build tools* → tria carpeta i nom.
2. Dins de `src`, crea `HolaMundo.java` amb el codi del costat.
3. Polsa el botó **Run ▶** (damunt del `main`) o prem `F5`.

HolaMundo.java

```java
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println(
            "¡Hola, Mundo! Porte anys esperant a que em creares."
        );
    }
}
```

```
> java HolaMundo
¡Hola, Mundo! Porte anys esperant
a que em creares.
```

⚠️ No confongues la **consola de l'IDE** amb la terminal del sistema: ací s'imprimeixen els `System.out.println` (pestanya Terminal / Eixida).

Dissecció d'Hola Món 🐸

1. `public class HolaMundo` — declares una classe. `public`: accessible des de fora; el nom ha de coincidir amb el de l'arxiu.
2. `public static void main(String[] args)` — el **botó d'inici**: Java busca esta línia i diu «per ací es comença!».
3. `System.out.println(...)` — **la veu** del programa: imprimeix el text en la consola.

ℹ️ Cada instrucció acaba amb **;** — el punt final de cada frase. Els **{ }** delimiten els blocs: els de la classe contenen la classe, els del `main` contenen les ordres.

La firma del main: paraula per paraula

Paraula	Què significa
public	Java pot trobar-lo des de fora: el botó és visible
static	Pot cridar-se sense necessitat de crear un objecte
void	No torna cap valor: fa el seu treball i es calla
main	El nom exacte que Java busca en arrancar. No en val un altre
String[] args	Una butxaca per a arguments en executar (Bloc 2)

⚠️ El `main` és **obligatori** i la seua firma exacta. Si el reanomenes `inici`, Java no troba la porta d'entrada i el programa no fa res.

print vs println

SaltsDeLinia.java

```java
System.out.print("Hola");
System.out.print("Mundo");
// ← sense salt final
System.out.println("Hola");
System.out.println("Mundo");
```

```
HolaMundo
Hola
Mundo
```

`print` imprimeix **sense** saltar de línia;
`println` imprimeix **i salta**.

ℹ️ Pensa com la JVM: executa **línia a línia, en orde**. Cada `;` tanca una frase; els `{ }`, un bloc.

El mètode que no es crida (trampa d'examen!)

Saludos.java

```java
public class Saludos {
    public static void main(String[] args) {
        System.out.println("¡Hola des del main!");
    }
    public static void saludo() {
        System.out.println("Açò mai s'executa...");
    }
}
```

Compila i s'executa bé... però **només imprimeix la primera línia**.

ℹ️ `saludo()` existix, però mai el crides des del `main`: es queda fent el vague. És un actor amb el guió aprés que mai ix a l'escenari.

🧠 **Truc:** el `main` és la porta d'entrada de la casa. Pot haver-hi moltes habitacions, però ningú entra per la finestra.

Tracta de pensar com el codi: tu ets la JVM

Computadora.java

```java
public class Computadora {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        int z = x + y;
        System.out.println("El resultado es: " + z);
    }
}
```

1. Trobe la classe i busque el `main`.
2. Cree `x` (5) i `y` (10).
3. Cree `z`, sume i guarde 15.
4. Cride: **«El resultado es: 15»**.

El depurador: visió de raigs X 🔍

El teu programa fa coses rares? No pegues a l'ordinador: usa el **depurador** (debugger).

🛑 **Breakpoint**: parar el programa en una línia concreta.

👣 **Step**: avançar instrucció a instrucció.

🔍 **Variables**: inspeccionar cada valor en cada moment.

✏️ **Editar**: canviar valors sobre la marxa.

ℹ️ És com vore una sèrie de crims en càmera lenta: pauses, observes qui fa què i analitzes cada detall. **El secret dels experts no és endevinar: és veure.**

Les quatre ferramentes del detectiu (VS Code)

Ferramenta	Drecera	Què fa
Breakpoint	Clic a l'esquerra del nº de línia	«Para ACÍ, vull vore què passa»
Step Over	F10	Executa la línia sense entrar en els detalls interns
Step Into	F11	Entra dins d'eixa crida: «vull espiar»
Variables / Watch	Panell lateral	«Ensenya'm el valor de la variable ARA MATEIX»

⚠️ Si entres sense voler en un mètode alié amb Step Into: **Step Out** (`Shift + F11`) et trau d'ací i et torna al punt de la crida.

ℹ️ Recorda: els errors de **compilació** els atrapa `javac` ; els de **lògica** compilen bé però fan el que no han de fer — per a eixos, el depurador és l'arma.

El cas del sospitós 🕵️

DetectivesDeCodigo.java

```java
public class DetectivesDeCodigo {
    public static void main(String[] args) {
        int sospechoso = 0;
        for (int i = 0; i < 10; i++) {
            sospechoso += i; // ← breakpoint ací
        }
        System.out.println("El culpable es: " + sospechoso);
    }
}
```

1. Breakpoint en `sospechoso += i` (clic a l'esquerra del número → punt roig).
2. Executa en mode depuració (`F5` / icona del bitxo).
3. La línia s'il·lumina en groc: observa el panell **Variables**.
4. F10 diverses vegades: veus canviar `sospechoso` i `i` en cada volta.

Què hauries de vore?

Valors de `sospechoso` en cada parada del bucle:

```
0, 1, 3, 6, 10, 15, 21, 28, 36, 45 — i en acabar: 55
> El culpable es: 55
```

✅ **La regla d'or:** quan alguna cosa falla, no endevines: observa. Reprodueix la fallada → breakpoint abans de la zona sospitosa → avança amb F10 → on es torça el valor, ací està el bug.

ℹ️ Si el programa no es deté mai: el breakpoint està en una línia que **no s'executa** (mètode no cridat, condició que no es complix). Una altra pista de detectiu.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //